Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology

School of Computing

Department of Computer Science and Engineering

10211CS224 / FULL STACK APPLICATION DEVELOPMENT (JAVA)

PROJECT REVIEW : MILESTONE PROJECT – 1

Date: 25-03-2026

Subscription-Based Membership Engine with Tiered Logic

SDG: SDG 08 – Decent Work and Economic Growth

Presented By:

Aadith Jagan Pamarthi VTU24437 23UECS0023

Course Handling Faculty:

Dr. S. Manohar M.E., (Ph.D.,)

TTS4009 - Assistant Professor

INTRODUCTION

Zen Gardens

1
2
3
4
5
6
7
8
9

- In today's digital landscape, tiered membership systems are essential for offering customized user experiences and features based on subscription levels.
- This project presents a full-stack, website that provides a tier-based subscription designed to manage user access, authentication, and permissions securely and efficiently.
- Built using Node.js, Express, and MySQL, it provides a API that talks to the database, while the frontend is crafted with standard HTML, CSS, and JavaScript.
- This system features secure user authentication with JSON Web Tokens (JWTs) and manages four distinct, inclusive membership tiers: Free, Bronze, Silver, and Gold.
- Key functionalities include real-time membership upgrades and robust access control, enforced on both the server-side and on the client-side to protect routes and content.
- Zen Gardens is a hobbyist website which provides an introduction to cultivating your own garden, provides equipment, and various levels of support based on your subscription tier.

PROBLEM STATEMENT

1
2
3
4
5
6
7
8
9

- Authentication, access control, and subscription management are often treated as disconnected systems, leading to complexity and security risks.
- This separation can create security gaps, allowing users to potentially access content or features beyond their subscribed tier.
- User session tokens frequently become outdated when a subscription changes mid-session, failing to reflect real-time permissions.
- Existing solutions often rely on third-party services, which obscures the underlying logic and reduces developer control and auditability.
- A self-contained system is required to unify authentication, enforce tier-based rules, and manage subscription plans in real-time, ensuring changes are reflected instantly without needing to re-login.

OBJECTIVE(S)

1
2
3
4
5
6
7
8
9

- To design and develop a secure, full-stack membership website that unifies user authentication, JWT-based session management, and tiered access control into a single, cohesive system.

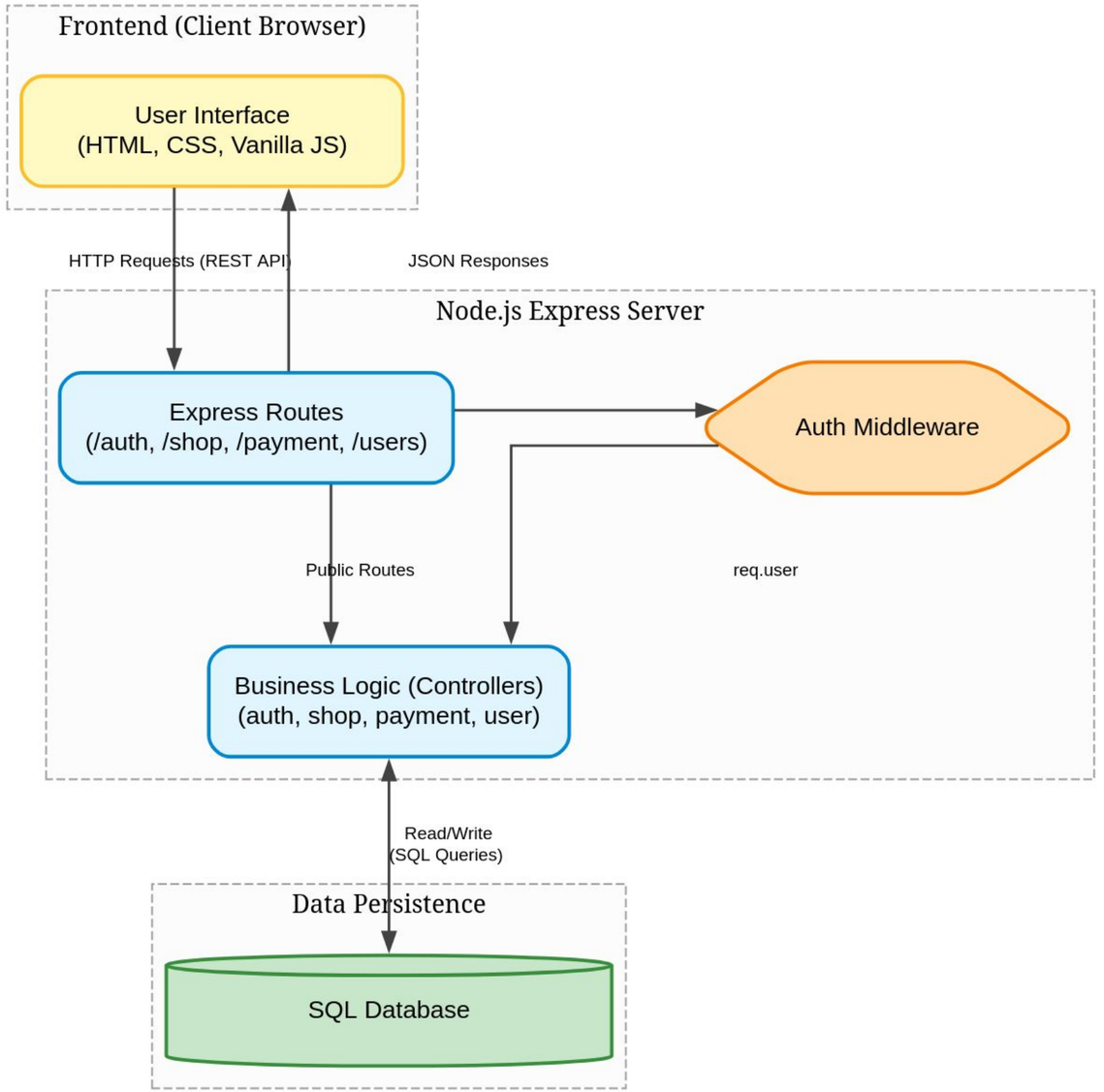
- To implement a real-time tier management workflow that allows users to instantly upgrade or downgrade their plan, with access rights synchronized immediately through token refreshes, eliminating the need for re-login.

PROPOSED SYSTEM

1
2
3
4
5
6
7
8
9

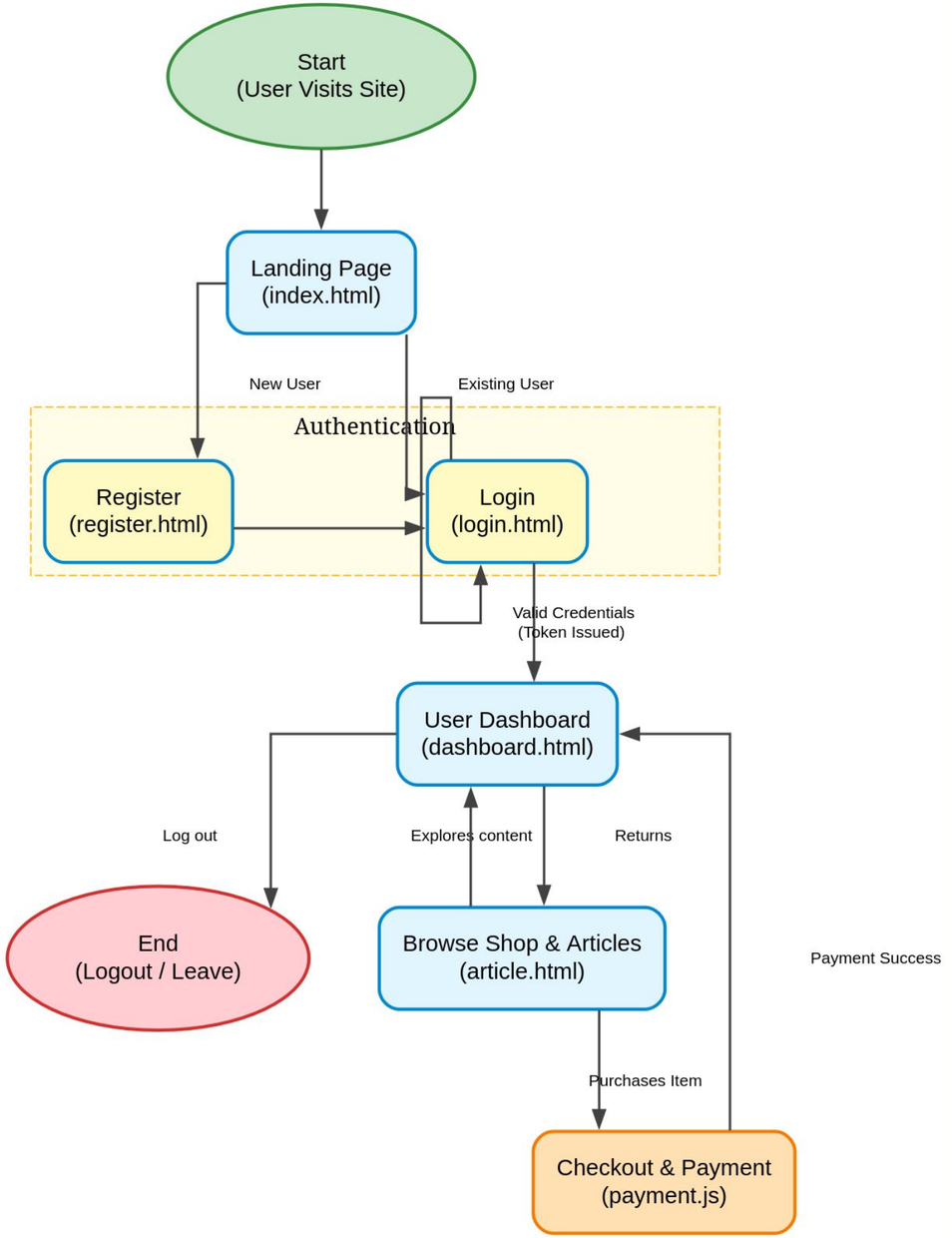
- A full-stack membership engine built on Node.js, Express, and MySQL that manages user registration, login, and tiered subscription plans through a unified REST API.
- Authentication is handled via JWT (json web token) issued on login and refreshed on tier changes, so the session always reflects the user's actual current plan without requiring re-login
- A four-tier inclusive membership model (Bronze -> Silver -> Gold ->Diamond) is enforced at the database level using a constrained ENUM, preventing invalid tier states entirely
- Every protected API route passes through a dedicated auth middleware that validates the JWT before any data is returned, keeping access control consistent and server-enforced
- The frontend uses a shared api.js session layer across all pages, eliminating duplicate auth logic and ensuring a single point of token storage, refresh, and logout

PROPOSED SYSTEM ARCHITECTURE



WORKFLOW

1
2
3
4
5
6
7
8
9



TECHNOLOGIES USED

1 2 3 4 5 6 7 8 9

Frontend:
HTML5, CSS3, Vanilla JavaScript;
Material 2 for UI components, Razorpay Test API for embedded payment handling

Backend:
Node.js, Express.js REST API; Razorpay Node SDK, bcryptjs for password hashing, jsonwebtoken (JWT) for authentication

Database:
MySQL relational schema (normalized tables for users, products, and order history), mysql2 Node.js driver

Tools:
Docker (MySQL container), Nodemon (live reload development), npm, browser DevTools, Razorpay Test Credentials Dashboard

Hardware / Software Requirements:

Requirement	Minimum
OS	Any (Linux / macOS / Windows)
Node.js	v18 or higher
MySQL	v8.0 or higher
RAM	2 GB
Browser	Any modern browser (Chrome, Firefox, Edge)

MODULE DESCRIPTION

Module 1: Authentication & Session Management

Handles user registration and login using bcrypt password hashing and JWT-based session tokens. On every subsequent request, tokens are validated server-side by the auth middleware before any protected resource is returned. Invalid or unauthorized requests are blocked before reaching business logic.

Module 2: Membership Tier Engine

Implements the core tiered access logic with four levels (Free, Bronze, Silver, Gold) enforced via a MySQL ENUM constraint. The module manages plan specifications and handles database updates when a user upgrades via the payment pipeline, ensuring their session token is refreshed to reflect their new tier immediately.

Module 3: E-commerce Shop & Marketplace

Manages the retrieval and purchase of digital shop items. Interfaces with normalized database tables (shop_items and shop_orders) to dynamically render the marketplace UI. Handles server-side validation to ensure authenticated users can seamlessly browse and securely purchase items.

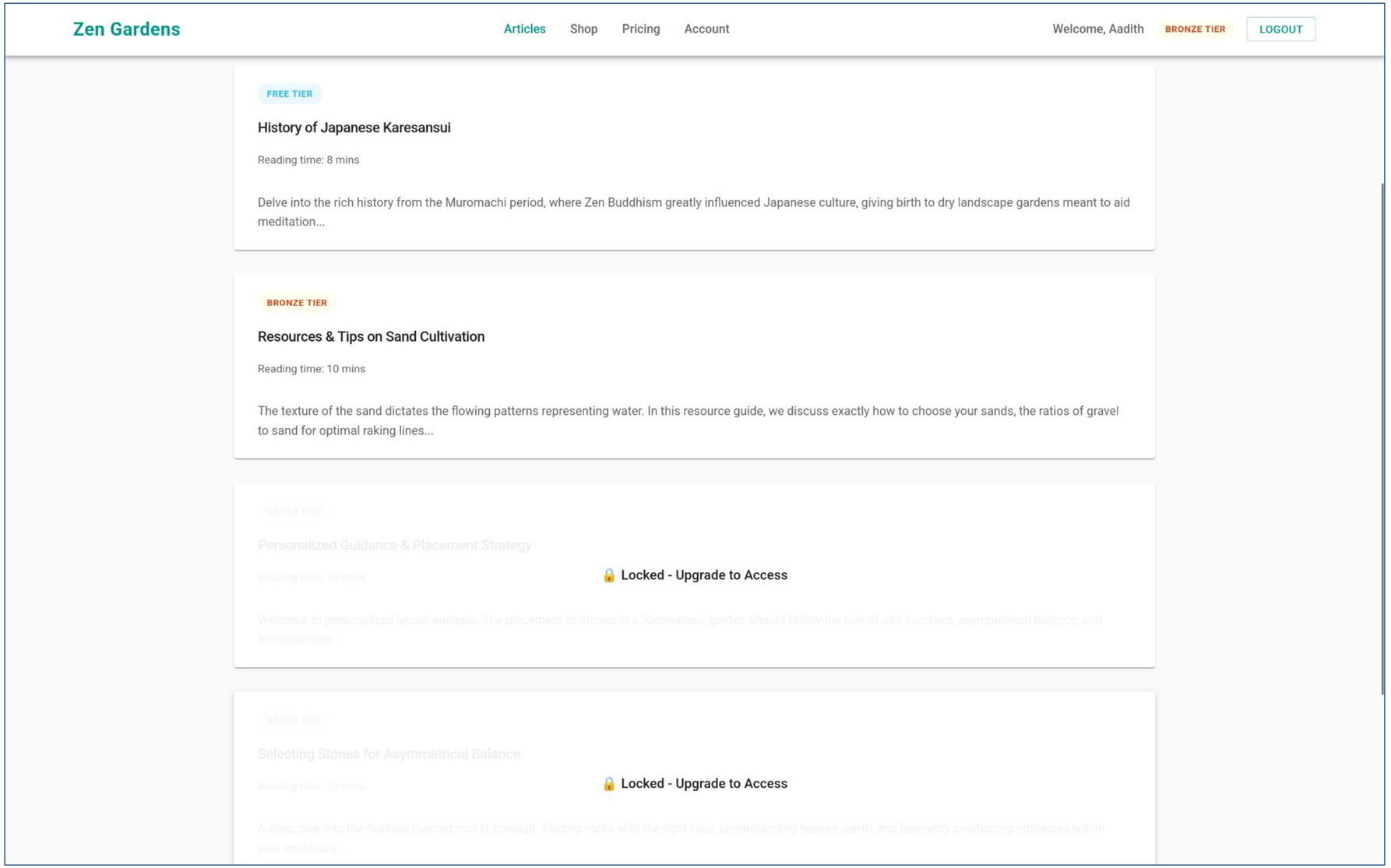
Module 4: Secure Payment Processing (Razorpay)

Coordinates financial transactions using the Razorpay Node.js SDK. Generates secure order IDs on the server, passes them to the client for embedded checkout, and strictly validates the Razorpay webhook signatures post-payment. Successful transactions automatically generate order receipts and update user access tiers.

Module 5: Frontend UI & Client Session Layer

A shared api.js module centralizes token storage and authenticated fetch calls across all pages. Each page enforces a strict client-side auth guard. UI components like tier badges, shop grids, and the Razorpay payment modal are rendered dynamically via Vanilla JavaScript without relying on heavy frontend frameworks.

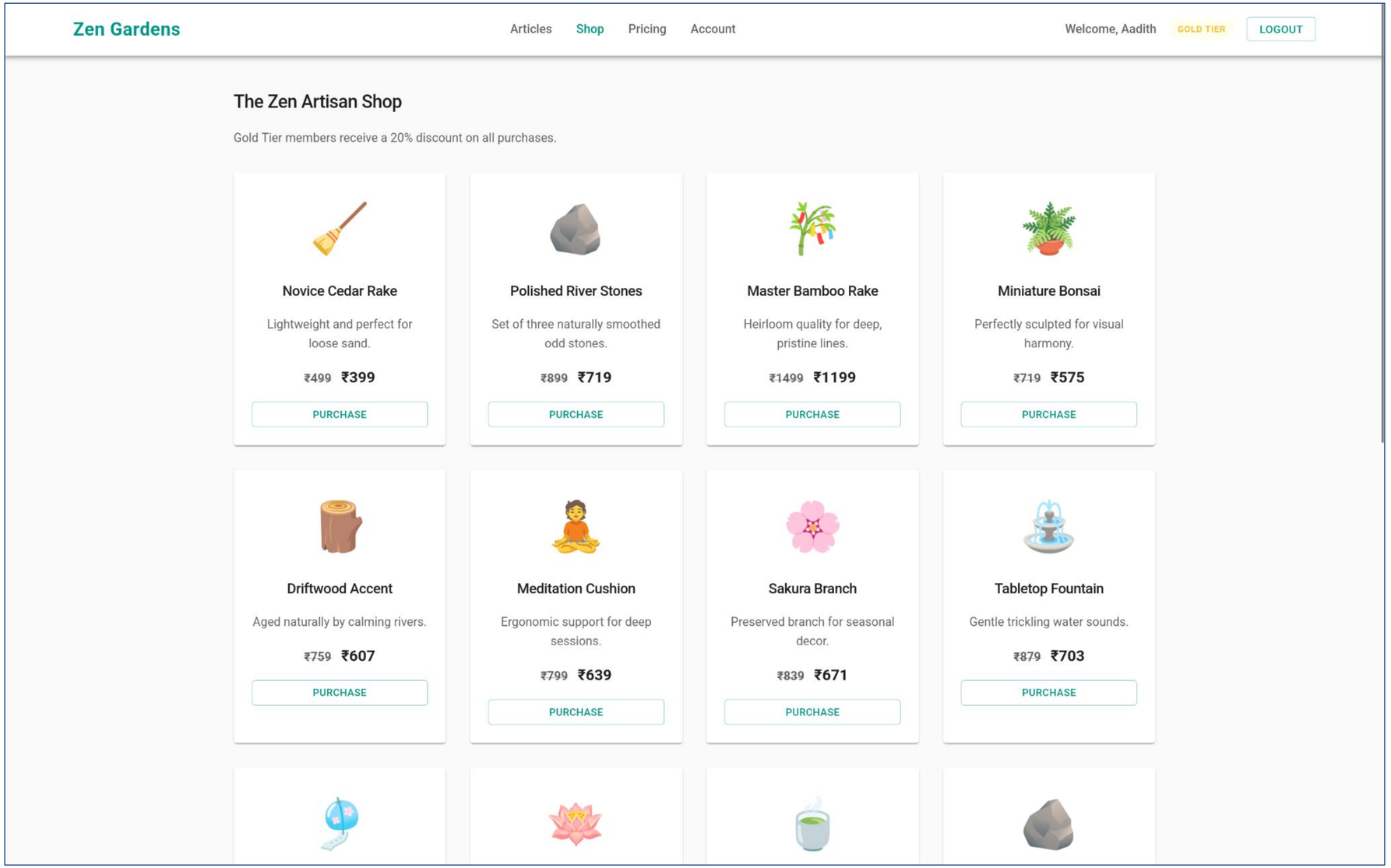
IMPLEMENTATION SCREENSHOTS



Home page of the Zen gardens website. As a bronze user, you have access to all articles that are bronze tier and below. For reading other articles, you need to subscribe to a better plan.

IMPLEMENTATION SCREENSHOTS

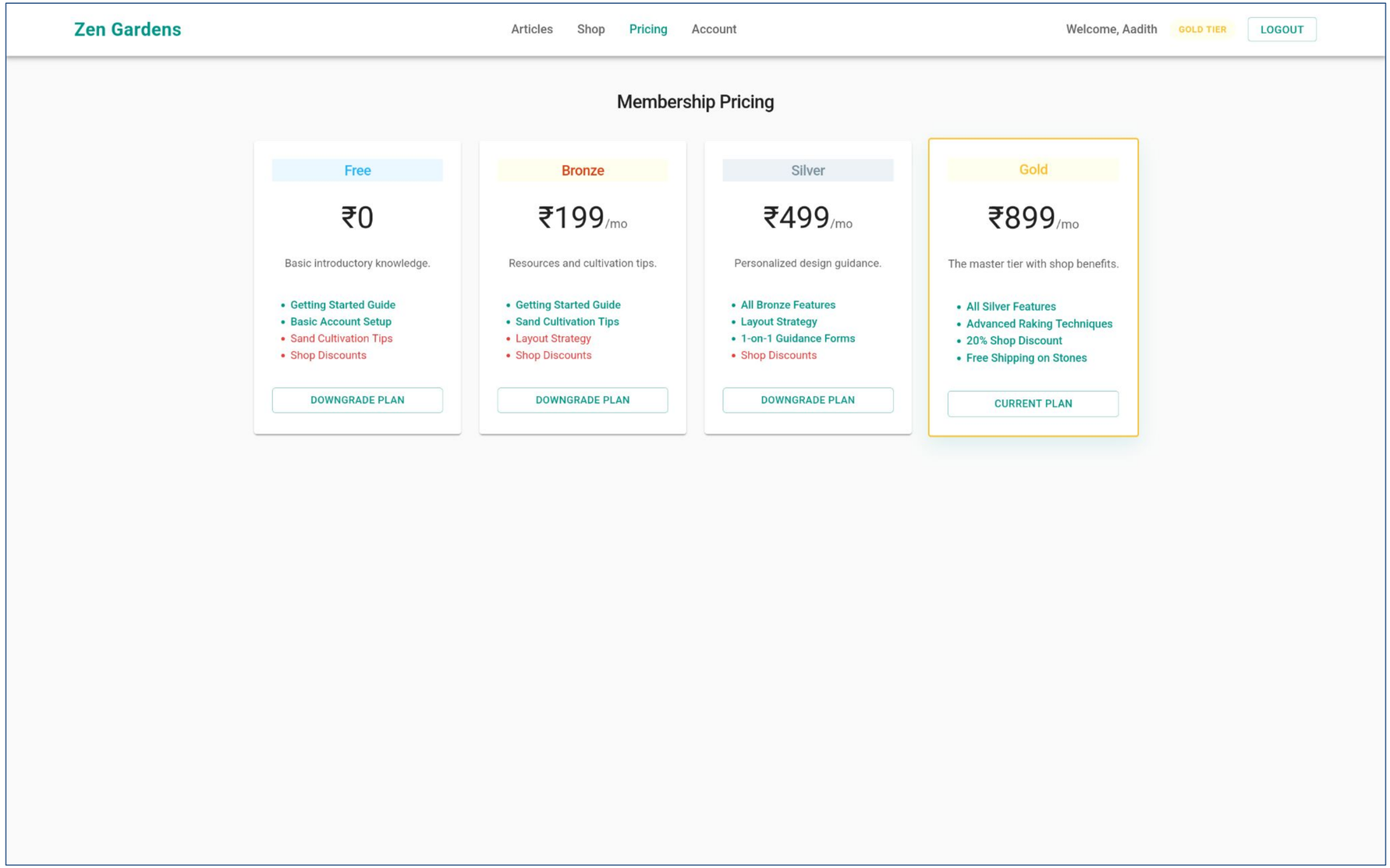
1
2
3
4
5
6
7
8(A)
9



The Zen Gardens shop webpage. Here, the user can purchase accessories and utilities for their zen garden and if you're a gold member, you get a 20% discount.

IMPLEMENTATION SCREENSHOTS

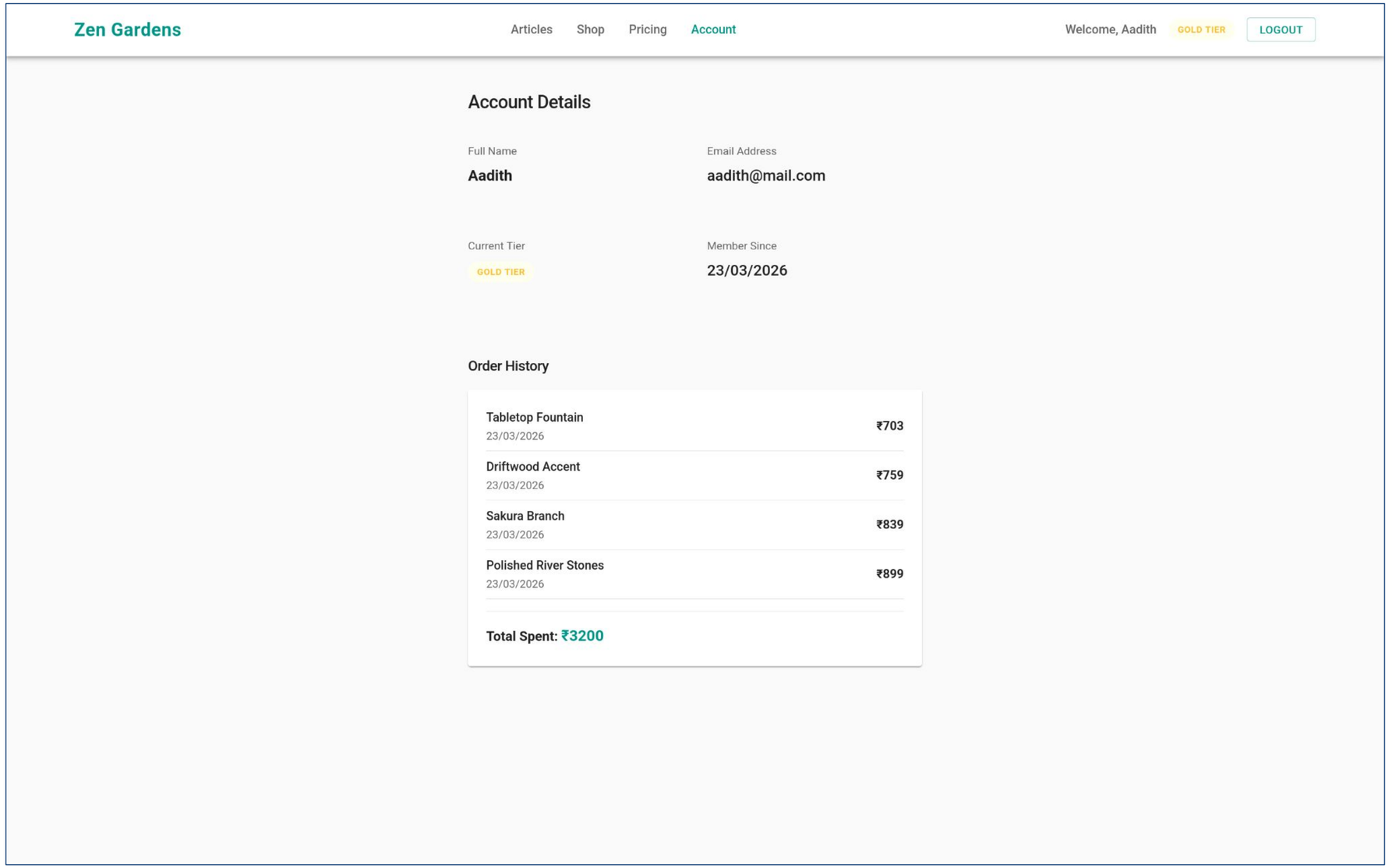
1
2
3
4
5
6
7
8(B)
9



The Pricing page of the Zen Gardens website. Here, you are shown the different pricing tiers that you can subscribe to. Each higher tier has inclusive benefits of the lower tier in addition to some additional benefits.

IMPLEMENTATION SCREENSHOTS

1
2
3
4
5
6
7
8(C)
9



Your account page in the Zen Gardens website. Here, your details are shown, and also the order history of your purchases from the shop.

ADVANTAGES & LIMITATIONS

1
2
3
4
5
6
7
8
9

Advantages

- The application now utilizes a normalized MySQL schema (separating users, shop_items, and orders). This eliminates the previous flat-schema limitations, allowing for complete transaction histories, item management, and future scalability.

- By integrating the Razorpay API, it enforces secure financial transactions through server-side order generation and strict webhook signature validation, preventing client-side manipulation.

- Successful transactions instantly update the database and issue a refreshed JWT in a single flow. This grants users immediate access to new tier perks without a re-login, all handled dynamically through a fast, maintainable Vanilla JavaScript frontend without heavy framework ove

Limitations

- JWTs are still stored in localStorage where they are accessible to JavaScript on the page. In a strict production environment, session management should be transitioned to secure HttpOnly cookies to mitigate Cross-Site Scripting (XSS) risks.

- While the system now tracks purchase history, it currently lacks automated recurring billing cycles. There are no automated background tasks or webhooks set up to handle time-based subscription expiration, trial periods, or automated tier downgrades.

Thank you